Programming Assignment 2

Data Prefetching and Cache Replacement in ChampSim

Team TeamName

Advay Bapat	23B1225
Aniket Gupta	23B1245
Apoorv Goyal	23B1266
Kundan Maiti	23B1242

Department of Electrical Engineering
Indian Institute of Technology Bombay

CS683: Advanced Computer Architecture

Date: October 14, 2026

Contents

1 Overview and Methodology . . . 2
1.1 Simulation Setup . . . 2
1.2 Configurations Evaluated . . . 2
1.3 Metrics . . . 2
2 Task 1: Data Prefetching . . . 3
2.1 Task 1A: Understanding the IP-Stride Prefetcher . . . 3
2.2 Task 1B: Stride Pattern Prefetcher (SPP) . . . 3
2.2.1 Design Overview . . . 3
2.2.2 Accuracy-Based Throttling . . . 4
2.2.3 Storage Overhead . . . 4
2.2.4 Results . . . 4
2.2.5 Discussion . . . 4
3 Task 2: Cache Replacement Policy . . . 6
3.1 Baseline: SRRIP at the L2 . . . 6
3.2 Task 2A: Instruction-Conscious RRIP (I-RRIP) . . . 6
3.2.1 Interface Change . . . 6
3.2.2 Policy Design . . . 6
3.2.3 Results . . . 7
3.3 Task 2B: Dynamic I-RRIP . . . 8
3.3.1 Set-Dueling Design . . . 8
3.3.2 Rationale for the Leader Sets and Decision Logic . . . 8
3.3.3 Sensitivity Study . . . 8
3.3.4 Results . . . 9
4 Task 3: SPP with Prefetch-Aware Dynamic I-RRIP . . . 10
4.1 Design . . . 10
4.2 Rationale for Handling Prefetched Lines . . . 10
4.3 Results . . . 10
5 Summary . . . 13
A Reproducing the Results . . . 13

1. Overview and Methodology

[Two or three sentences: what was implemented in each task and the headline result (best geomean speedup and which configuration achieved it).]

1.1 Simulation Setup

All experiments use the ChampSim simulator from the PA2 repository, with 25M warm-up instructions followed by 25M simulated instructions on each of the three provided traces (`Trace1`, `Trace2`, `Trace3`). Table 1 lists the simulated system.

Table 1. Simulated single-core system.

Component	Configuration
Branch predictor	Hashed perceptron
L1I cache	32 KB, 64 sets × 8 ways, 4-cycle latency
L1D cache	48 KB, 64 sets × 12 ways, 5-cycle latency, 16 MSHRs
L2 cache	512 KB, 1024 sets × 8 ways, 10-cycle latency, 32 MSHRs
LLC	2 MB, 2048 sets × 16 ways, 20-cycle latency
Block / page size	64 B / 4 KB
Warm-up / simulation	25M / 25M instructions

1.2 Configurations Evaluated

Table 2. Binaries built for this report (`./build_champsim.sh <L1D prefetcher> <L2C replacement> <name>`).

Name	L1D prefetcher	L2C replacement	Used in
Baseline	`no`	`lru`	All tasks
IP-stride	`ip_stride`	`lru`	Task 1
SPP	`spp`	`lru`	Tasks 1, 3
RRIP	`no`	`rrip`	Task 2
I-RRIP	`no`	`irrip`	Task 2A
Dyn. I-RRIP	`no`	`dyn_irrip`	Tasks 2B, 3
SPP + Dyn. I-RRIP	`spp`	`dyn_irrip`	Task 3

1.3 Metrics

$$\text{Speedup} = \frac{\text{IPC with optimisation}}{\text{IPC of baseline (no prefetching)}} \tag{1}$$

$$\text{L1D Coverage} = \frac{\text{L1D load misses}_{\text{baseline}} - \text{L1D load misses}_{\text{prefetcher}}}{\text{L1D load misses}_{\text{baseline}}} \tag{2}$$

$$\text{Prefetch Accuracy} = \frac{\text{useful prefetches}}{\text{useful} + \text{useless prefetches}} \tag{3}$$

$$\text{MPKI} = \frac{\text{load misses}}{\text{instructions}/1000} \tag{4}$$

MPKI and prefetch accuracy are taken directly from the ChampSim statistics. Averages across traces are reported as geometric means for speedup and as arithmetic means for MPKI, accuracy and coverage.

2. Task 1: Data Prefetching

2.1 Task 1A: Understanding the IP-Stride Prefetcher

[Explain `prefetcher/ip_stride.l1d_pref`: the tracker table and its fields, how an entry is indexed by IP, how the stride is learned and confirmed, when and how many prefetches are issued, and how the page boundary is handled.]

Tracker table. *[Number of entries, indexing, replacement of tracker entries.]*

Training. *[How the last address and last stride are updated on every access.]*

Prefetch issue. *[Condition for issuing prefetches, the degree, and the page-boundary check.]*

Limitations. *[Why tracking one last stride per IP cannot capture a repeating sequence of strides]*, e.g. $(+1, +2, +2, +2, +3)^n$. *[This motivates SPP.]*

2.2 Task 1B: Stride Pattern Prefetcher (SPP)

2.2.1 Design Overview

[High-level description of the prefetcher. A block diagram (figures/spp_diagram) is recommended.]

Plot: `figures/spp_diagram.pdf / .png`

Figure 1. Organisation of the Stride Pattern Prefetcher at the L1D.

IP table. *[Per-IP state tracked: tag, last address, last stride, confidence, signature, mode.]*

Uniform-stride detection. *[How a constant stride is detected and confirmed.]*

Fallback to non-uniform mode. *[The confidence counter / state machine that detects that the uniform pattern has broken and switches the IP to signature-based prediction, and how it switches back.]*

Signature and pattern table. *[How the signature is built from recent strides (e.g. shift-and-XOR of the last k strides), its width, how it indexes the pattern table and what the table predicts.]*

Multi-degree prefetching. *[How prefetches beyond the first are generated (stride repeated for uniform mode; signature walked forward for non-uniform mode).]*

Page boundary. Every candidate address is checked against the 4 KB page of the triggering access, and prefetching stops at the first candidate that falls outside it. *[Confirm / adapt.]*

Key routine (`prefetcher/spp.l1d_pref`)

```
// TODO: paste the core of l1d_prefetcher_operate() here
```

2.2.2 Accuracy-Based Throttling

[How useful and issued prefetches are counted, the length of an evaluation epoch, and how the degree is changed at the end of each epoch.]

Table 3. Prefetch degree chosen for each accuracy range.

Accuracy range	Degree	Reason
acc $\geq$ –	–	*[high accuracy: be aggressive]*
– $\leq$ acc $<$ –	–	*[. . .]*
– $\leq$ acc $<$ –	–	*[. . .]*
acc $<$ –	–	*[low accuracy: avoid pollution]*

How the thresholds were chosen. *[Deliverable: what thresholds were used and how they were decided (sweep, per-trace accuracy distribution, etc.). Refer to any sweep data.]*

Why these degrees. *[Deliverable: justify the degree in each accuracy range (Table 3).]*

2.2.3 Storage Overhead

[Deliverable: exact bit-level breakdown of every table entry. Edit the fields to match your implementation.]

2.2.4 Results

Table 5 compares the baseline, the provided IP-stride prefetcher and our SPP; Figures 2 to 5 plot the same data.

Plot: `figures/task1_speedup.pdf` / .png

Figure 2. Speedup over the no-prefetching baseline.

2.2.5 Discussion

[Explain the results trace by trace: which traces have uniform vs. non-uniform patterns, where SPP gains over IP-stride and why, effect of throttling on accuracy, any trace where prefetching hurts.]

Table 4. Bit-level storage breakdown of the SPP.

Structure	Field	Bits/entry	Entries	Total bits
IP table	Valid	1	–	–
	IP tag	–	–	–
	Last line offset in page	6	–	–
	Last stride (signed)	7	–	–
	Stride confidence	–	–	–
	Mode (uniform / non-uniform)	1	–	–
	Signature	–	–	–
	Replacement (LRU) bits	–	–	–
Pattern table	Predicted stride	7	–	–
	Confidence	–	–	–
Throttling	Issued counter	–	1	–
	Useful counter	–	1	–
	Current degree	–	1	–
Total				**– bits (– bytes)**

Table 5. Task 1 results (L1D prefetching, LRU at L2).

Trace	Config	IPC	Speedup	L1D load MPKI	Accuracy (%)	Coverage (%)
Trace 1	Baseline	–	1.000	–	–	–
	IP-stride	–	–	–	–	–
	SPP	–	–	–	–	–
Trace 2	Baseline	–	1.000	–	–	–
	IP-stride	–	–	–	–	–
	SPP	–	–	–	–	–
Trace 3	Baseline	–	1.000	–	–	–
	IP-stride	–	–	–	–	–
	SPP	–	–	–	–	–
Mean	IP-stride	–	–	–	–	–
	SPP	–	–	–	–	–



Figure 3. L1D load MPKI.



Figure 4. Prefetch accuracy.



Figure 5. L1D coverage.

3. Task 2: Cache Replacement Policy

3.1 Baseline: SRRIP at the L2

The provided `replacement/rrip.l2c_repl` implements SRRIP with a 2-bit re-reference prediction value (RRPV, maxRRPV = 3):

- **Insertion:** a newly filled line gets RRPV = 2 (long re-reference interval).
- **Promotion:** a hit sets RRPV = 0 (near-immediate re-reference).
- **Eviction:** the first way with RRPV = 3 is chosen; if none exists, all RRPVs in the set are incremented and the search repeats.

3.2 Task 2A: Instruction-Conscious RRIP (I-RRIP)

3.2.1 Interface Change

[Describe how `update_replacement_state` was extended with an `isInstruction` argument: the declaration in `inc/cache.h`, every call site in `src/cache.cc`, and how the value is taken from `packet.instruction`.]

Extended signature (`inc/cache.h`)

```
void l2c_update_replacement_state(uint32_t cpu, uint32_t set, uint32_t way,
        uint64_t full_addr, uint64_t ip, uint64_t victim_addr,
        uint32_t type, uint8_t hit, uint8_t isInstruction);
```

3.2.2 Policy Design

Table 6 summarises how I-RRIP differs from SRRIP in each of the three parts of the policy.

Insertion. *[Rationale for modifying or not modifying insertion.]*

Table 6. Insertion, promotion and eviction in SRRIP and I-RRIP.

Part	SRRIP	I-RRIP
Insertion	RRPV = 2 for every line	*[instruction lines: . . . ; data lines: . . .]*
Promotion	RRPV = 0 on any hit	*[. . .]*
Eviction	First line with RRPV = 3; age set otherwise	*[. . .]*

Promotion. *[Rationale for modifying or not modifying promotion.]*

Eviction. *[Rationale for modifying or not modifying eviction.]*

> **Why this design is the best of those we tried.** *[Deliverable: defend the choice. List the alternatives evaluated (e.g. instruction-only promotion, data-first eviction, different insertion RRPVs) and the evidence against each.]*

3.2.3 Results

Table 7. Task 2A results (no prefetcher).

Trace	L2 policy	IPC	Speedup	L2C load MPKI	L2 instr. MPKI	L2 data MPKI
Trace 1	LRU	–	1.000	–	–	–
	SRRIP	–	–	–	–	–
	I-RRIP	–	–	–	–	–
Trace 2	LRU	–	1.000	–	–	–
	SRRIP	–	–	–	–	–
	I-RRIP	–	–	–	–	–
Trace 3	LRU	–	1.000	–	–	–
	SRRIP	–	–	–	–	–
	I-RRIP	–	–	–	–	–
Mean	SRRIP	–	–	–	–	–
	I-RRIP	–	–	–	–	–

Plot: `figures/task2a_speedup.pdf` / .png

Figure 6. Task 2A: speedup of SRRIP and I-RRIP.



Figure 7. Task 2A: L2C, L2 instruction and L2 data load MPKI.

Discussion. *[Change in L2 instruction MPKI vs. data MPKI per trace, and the resulting IPC change. Note any trace where data misses rise enough to hurt.]*

3.3 Task 2B: Dynamic I-RRIP

3.3.1 Set-Dueling Design

[Describe the leader sets (how many, how they are chosen), what is counted in them (instruction and total accesses and misses), how often the counters are evaluated / reset, and how follower sets pick a policy.]

The decision metric is

$$r = \frac{\text{instr. accesses} + \text{instr. misses}}{\text{total accesses} + \text{total misses}}, \qquad \text{policy} = \begin{cases} \text{I-RRIP} & r > \theta \\ \text{SRRIP} & \text{otherwise} \end{cases} \tag{5}$$

with threshold $\theta = -$. *[Edit if you changed the formula or added parameters.]*

Policy selection (`replacement/dyn_irrip.l2c_repl`)

```
// TODO: paste the leader-set check and decision logic here
```

3.3.2 Rationale for the Leader Sets and Decision Logic

Number of leader sets. *[Deliverable: why this number (sampling accuracy vs. the cost of running leader sets on the non-preferred policy), and how they are spread over the 1024 sets.]*

Decision logic. *[Deliverable: why this metric, why accesses and misses are combined, epoch length / hysteresis.]*

3.3.3 Sensitivity Study

[Deliverable: sensitivity to the decision parameters (at least 5 threshold values).]

Table 8. Sensitivity of Dynamic I-RRIP to the threshold θ (speedup over LRU).

θ	Trace 1	Trace 2	Trace 3	Geomean	Mean L2 instr. MPKI
–	–	–	–	–	–
–	–	–	–	–	–
–	–	–	–	–	–
–	–	–	–	–	–
–	–	–	–	–	–

[Discuss the trend and the value chosen. Optional: a second sweep over the number of leader sets.]



Figure 8. Speedup of Dynamic I-RRIP against the threshold θ.

3.3.4 Results

Table 9. Task 2B results (no prefetcher, $\theta = -$).

Trace	L2 policy	IPC	Speedup	L2C load MPKI	L2 instr. MPKI	L2 data MPKI
	SRRIP	–	–	–	–	–
Trace 1	I-RRIP	–	–	–	–	–
	Dyn. I-RRIP	–	–	–	–	–
	SRRIP	–	–	–	–	–
Trace 2	I-RRIP	–	–	–	–	–
	Dyn. I-RRIP	–	–	–	–	–
	SRRIP	–	–	–	–	–
Trace 3	I-RRIP	–	–	–	–	–
	Dyn. I-RRIP	–	–	–	–	–
Mean	Dyn. I-RRIP	–	–	–	–	–



Figure 9. Task 2B: speedup of SRRIP, I-RRIP and Dynamic I-RRIP.

Discussion. *[Fraction of time each trace spends in I-RRIP mode, and where dynamic switching recovers data misses that static I-RRIP lost.]*



Figure 10. Task 2B: L2C, L2 instruction and L2 data load MPKI.

4. Task 3: SPP with Prefetch-Aware Dynamic I-RRIP

4.1 Design

[How `dyn_irrip.l2c_repl` treats lines whose packet type is `PREFETCH`: insertion RRPV for prefetched lines, what happens on the first demand hit to a prefetched line, whether prefetch hits promote, and any change to eviction priority or to the set-dueling counters.]

Table 10. Treatment of each line class in the prefetch-aware policy.

Line class	Insertion	Promotion	Eviction priority
Instruction (demand)	*[. . .]*	*[. . .]*	*[. . .]*
Data (demand)	*[. . .]*	*[. . .]*	*[. . .]*
Prefetched	*[. . .]*	*[. . .]*	*[. . .]*

4.2 Rationale for Handling Prefetched Lines

[Deliverable: why prefetched lines are handled this way (e.g. inaccurate prefetches should not displace instruction lines; accurate prefetches should survive until their demand use; prefetch fills must not bias the set-dueling counters).]

4.3 Results

Table 11. Task 3 results: performance and prefetch metrics.

Trace	Config	IPC	Speedup	Accuracy (%)	Coverage (%)	L1D MPKI
	SPP	–	–	–	–	–
Trace 1	Dyn. I-RRIP	–	–	–	–	–
	SPP + Dyn. I-RRIP	–	–	–	–	–
	SPP	–	–	–	–	–
Trace 2	Dyn. I-RRIP	–	–	–	–	–
	SPP + Dyn. I-RRIP	–	–	–	–	–
	SPP	–	–	–	–	–
Trace 3	Dyn. I-RRIP	–	–	–	–	–
	SPP + Dyn. I-RRIP	–	–	–	–	–
Mean	SPP + Dyn. I-RRIP	–	–	–	–	–

Table 12. Task 3 results: L2 load MPKI.

Trace	Config	L2C load MPKI	L2 instr. MPKI	L2 data MPKI
Trace 1	SPP	–	–	–
	Dyn. I-RRIP	–	–	–
	SPP + Dyn. I-RRIP	–	–	–
Trace 2	SPP	–	–	–
	Dyn. I-RRIP	–	–	–
	SPP + Dyn. I-RRIP	–	–	–
Trace 3	SPP	–	–	–
	Dyn. I-RRIP	–	–	–
	SPP + Dyn. I-RRIP	–	–	–

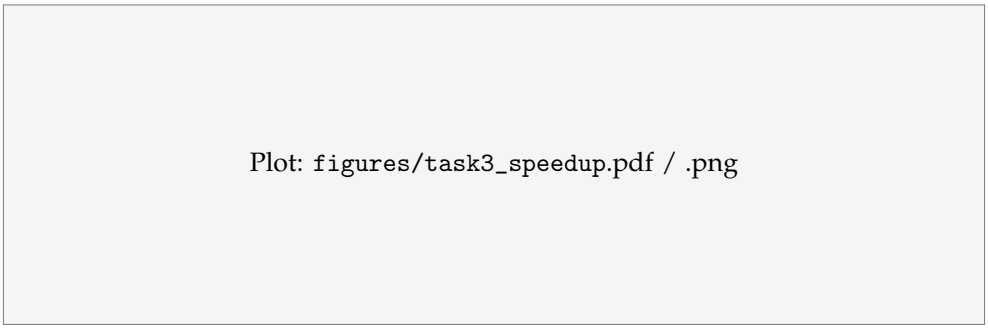


Figure 11. Task 3: speedup of SPP, Dynamic I-RRIP and their combination.

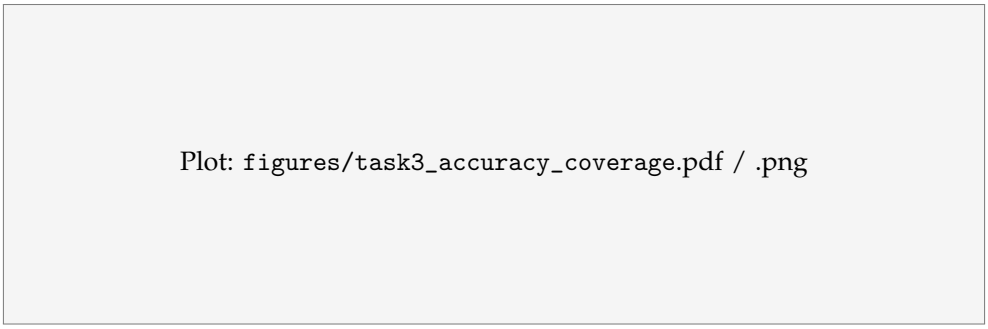


Figure 12. Task 3: prefetch accuracy and coverage.



Figure 13. Task 3: L1D load MPKI.

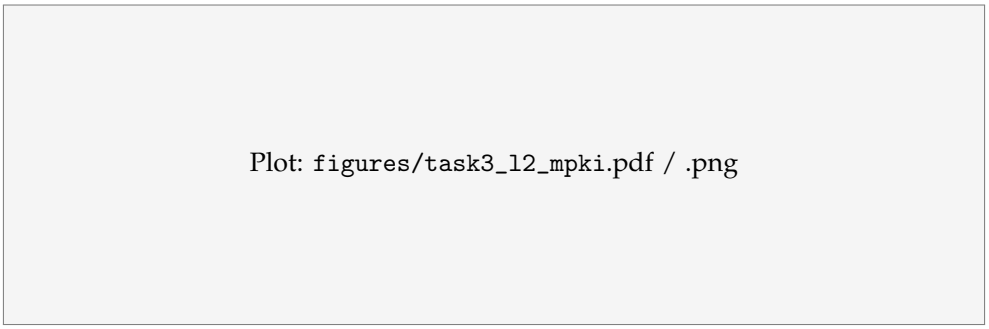


Figure 14. Task 3: L2C, L2 instruction and L2 data load MPKI.

Discussion. *[Show that the combination outperforms both standalone SPP and Dynamic I-RRIP in IPC, and explain where the extra gain comes from.]*

5. Summary

Table 13. Geometric-mean speedup of every configuration over the baseline.

Configuration	Trace 1	Trace 2	Trace 3	Geomean
IP-stride	–	–	–	–
SPP	–	–	–	–
SRRIP	–	–	–	–
I-RRIP	–	–	–	–
Dyn. I-RRIP	–	–	–	–
SPP + Dyn. I-RRIP	–	–	–	–



Figure 15. Speedup of all configurations.

[Short closing paragraph: key takeaways from each task.]

A. Reproducing the Results

Build and run

```
./build_champsim.sh no lru baseline
./build_champsim.sh spp dyn_irrip spp_dynirrip
./bin/<binary> -warmup_instructions 25000000 \
    -simulation_instructions 25000000 -traces Trace_Files/Trace1.gz
```

[Describe the run and result-parsing scripts in the repository and where the raw outputs are stored.]